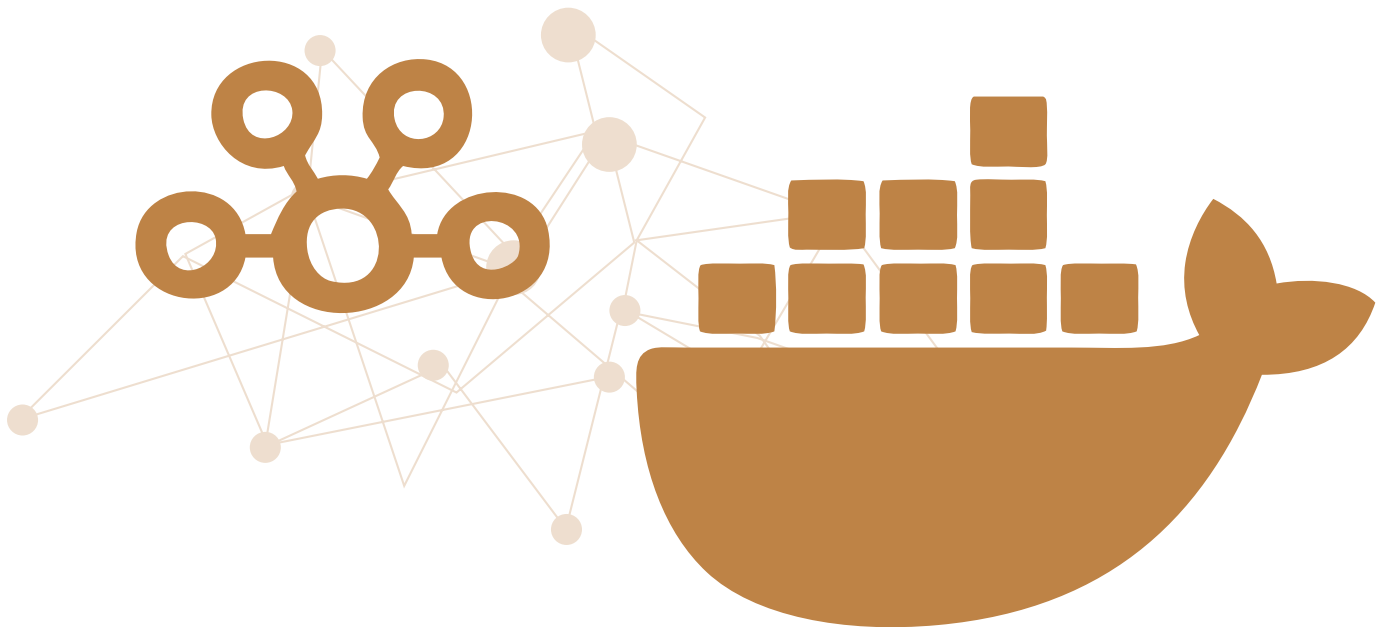


Microservices Collaboration with Containerised Kafka

Independently deployed microservices need a mechanism to collaborate among themselves asynchronously. Apache Kafka has emerged as the first choice for establishing such an environment. In this part 8 of the series of articles titled 'The Design Odyssey', the *AddService* of the user management system is connected to Kafka on the Docker network.



Apache Kafka plays different roles in different use cases. It can act as a message store, a messaging queue, or a stream processing engine. In the enterprise microservices environment, it has been very popular as a messaging queue where the services communicate with each other by producing and consuming messages asynchronously. Apache Kafka makes sure that the messages are delivered from the producers to the consumers with various guarantees. (Refer to the article published in the September 2021 issue of *Open Source For You* for an introduction to Apache Kafka by the author.)

The path covered so far

Before introducing Kafka into the architecture of the user management system (UMS), let's look at the big picture once.

We initially designed the UMS as a monolith system by applying appropriate object-oriented design patterns like Factory, Adapter, Proxy, Observer, etc. Later, we

refactored the design by decomposing the monolith into a set of microservices, namely, *AddService*, *SearchService*, *FindService*, and *JournalService* by applying the principles of Domain-Driven Design. We chose to develop REST API for the *AddService* on the Spring Boot platform and integrated it with a JPA database. And in the previous part, the *AddService* was containerised and deployed on the Docker platform.

We are yet to integrate the *AddService* with the *JournalService*.

Recollect that the job of the *JournalService* is to log all the activities on UMS into a central location without being a show-stopper. In other words, the *JournalService* must listen to the events published by the *AddService*, *FindService* as well as *SearchService*. Obviously, it requires an implementation of the observer pattern. Apache Kafka is one such implementation when it is used as a messaging infrastructure.